PROJET FINAL · PYTHON AVANCÉ · ING3

Analyser 12 Go de logs sans faire exploser la RAM

Engine de traitement et d'analyse de flux de données sécurisé

12,1 Go	69,3 M	210 834	48
de logs générés	de lignes analysées	alertes de sécurité	attaquants démasqués

AU PROGRAMME

1. **Module 1** — Ingestion & générateurs
2. **Module 2** — Pipeline fonctionnel
3. **Module 3** — Agrégation & détection
4. **Module 4** — Règles & introspection

L'ÉQUIPE

NAYME Yanis

GUEYE Babacar Mansour

CLABAU Brandon

LYOUBI Mehdi

Enseignant : JOUD HANAY

https://github.com/Babacar2804/Projet-PythonSecure-log-anlysis

Soutenance · 12/11/2026

OUVERTURE

0 Le projet en bref

Le sujet, nos données et le chemin d'une ligne de log

Le sujet — une plateforme en ligne de commande qui **ingère, nettoie et analyse** des journaux de logs volumineux **sans jamais les charger entièrement en mémoire**. Pour l'éprouver, notre générateur simule 30 jours d'activité d'un site e-commerce : 11 services, des pannes, un DDoS, des attaques, et ~5 % de lignes volontairement abîmées.

12,1 Go
générés en 75 s par 7 processus

69,3 M
lignes dans 12 fichiers

17 538
adresses IP différentes

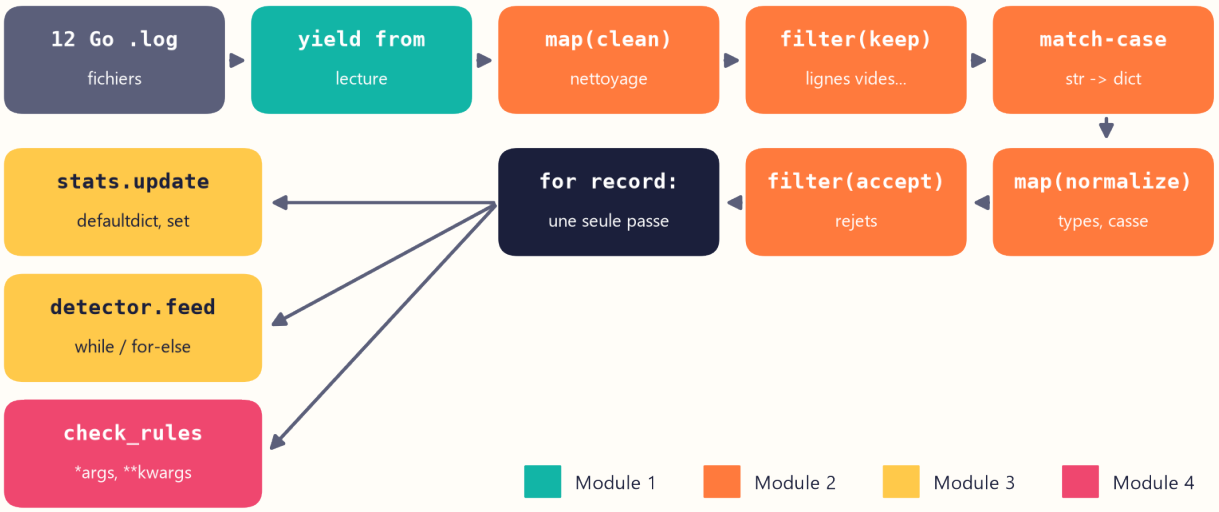
17 min
pour tout analyser, en une seule passe

Trois formats de lignes bien structurés

data/raw/logs-00-20261001-0000.log

```
# applicatif (80 %) : 7 champs séparés par |
2026-10-01T08:15:32.481Z | INFO | api-gateway | 10.0.3.17:51432/TCP | user=u1042 | event=http_request | {...}
# JSON (10 %)
{"ts":"2026-10-01T08:15:32Z","lvl":"error","svc":"payment-service","client":{"ip":...},"code":502}
# pare-feu (10 %) : champs positionnels
FW 2026-10-01T00:20:12.423Z DROP TCP 102.109.77.236 27409 10.0.100.29 22 DENY-SSH-EXT
```

Le chemin d'une ligne : chaque couleur = un module



TRANSITION Tout commence par la lecture des fichiers, sans les charger en mémoire → Module 1

MODULE 1 · CE QU'ON PRÉSENTE

1 Ingestion & générateurs

Lire 12 Go une ligne à la fois : yield, yield from, parsing en dictionnaires

CE QUE DEMANDE L'ÉNONCÉ

- Un générateur basé sur `yield` pour lire un fichier de logs ligne par ligne, à la demande.
- Un sous-générateur qui lit plusieurs fichiers successifs avec `yield from`.
- Chaque ligne parsée en dictionnaire avec des types variés : horodatage, niveau, IP, utilisateur, payload JSON ou chaîne brute.
- Des tuples pour les métadonnées fixes : (IP, Port, Protocole).

Ce qu'on présente, dans l'ordre

1. **Le problème** — 12 Go et 69 millions de lignes ne tiennent pas en mémoire. On ne lit donc jamais un fichier en entier : pas de `read()`, pas de `readlines()`.
2. **yield** — `read_log_file` renvoie une ligne puis se met en pause ; elle reprend au même endroit quand la boucle demande la suivante. Le `with` garde le fichier ouvert jusqu'au bout.
3. **yield from** — `read_sources` enchaîne les 12 fichiers comme un seul flux ; un fichier manquant est signalé puis ignoré (`try / except`).
4. **Robustesse** — `errors="replace"` : un octet invalide ne fait pas planter la lecture ; `newline=""` garde les `\r` pour qu'on les nettoie nous-mêmes.
5. **Dictionnaire typé** — chaque ligne devient un `LogRecord` (`TypedDict`) : `datetime`, niveau, service, IP, utilisateur, payload JSON ou texte brut.
6. **Tuple** — la connexion est stockée dans le tuple immuable `(IP, Port, Protocole)` : on ne peut pas la modifier par erreur, et elle peut servir de clé (exploité au module 3).

Le code à montrer (logengine/ingestion.py)

ingestion.py

```python
def read_log_file(path: Path) -> Iterator[str]:
    with open(path, encoding="utf-8", errors="replace", newline="") as file:
        for line in file:
            yield line          # pause : une seule ligne en mémoire

def read_sources(paths: Iterable[Path]) -> Iterator[str]:
    for path in paths:
        try:
            yield from read_log_file(path)   # délègue au sous-générateur
        except (FileNotFoundError, PermissionError) as exc:
            print(f"fichier ignoré : {path}", file=sys.stderr)
```

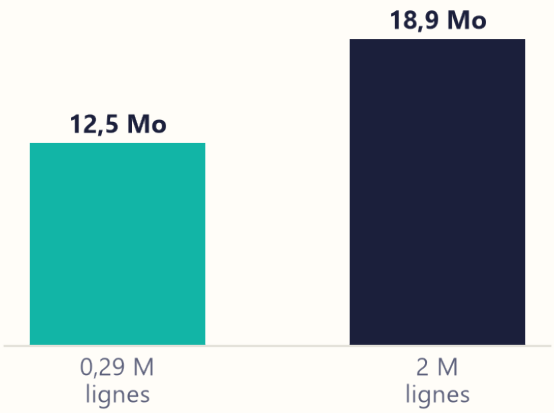
MODULE 1 · RÉSULTATS

1 Ingestion : ce que ça donne

La preuve par la mémoire, et le bonus temps réel

69,3 M	69 204	18,9 Mo	48 o
lignes lues une à une	lignes par seconde	de pic mémoire pour 2 M de lignes	taille de l'objet `records` (audit)

La mémoire ne suit pas le volume



Bonus : le temps réel

follow_log_file (tail -f)

```
while time.monotonic() < deadline:
    chunk = file.readline()
    if not chunk:                # rien de neuf
        time.sleep(0.2)
        continue
    pending += chunk
    if not pending.endswith("\n"):
        continue                 # ligne incomplète
    yield pending
    pending = ""
```

Générateur **infini** : il attend au lieu de s'arrêter. `python main.py live` l'utilise avec **le même pipeline** que les fichiers.

QUESTIONS PROBABLES DU JURY

Q **Pourquoi un générateur plutôt qu'une liste ?**
Une liste garde tous les éléments en mémoire en même temps : 12 Go feraient planter le programme. Le générateur produit une ligne à la demande, puis l'oublie.

Q **Quelle différence entre yield et return ?**
return termine la fonction. yield la met en pause en gardant son état : elle reprend juste après, à l'appel suivant.

Q **Et si on écrivait list(records) ?**
Toutes les lignes seraient chargées en RAM. Et un générateur ne se parcourt qu'une fois : une deuxième boucle ne trouverait rien.

Q **Comment prouvez-vous que la RAM ne sature pas ?**
Avec tracemalloc : 12,5 Mo pour 290 699 lignes, 18,9 Mo pour 2 M : sept fois plus de données, à peine 1,5 fois plus de mémoire.

Q **Pourquoi un tuple pour la connexion ?**
Immuable, donc protégé ; hashable, donc utilisable comme clé de dictionnaire ou dans un set (une liste ne l'est pas).

TRANSITION Les lignes arrivent une par une : il faut maintenant les nettoyer et les comprendre → Module 2

MODULE 2 · CE QU'ON PRÉSENTE

2 Pipeline fonctionnel

filter, map, lambda et match-case : nettoyer et structurer chaque ligne

CE QUE DEMANDE L'ÉNONCÉ

- `filter()` avec des `lambda` pour exclure les requêtes invalides et les entrées vides (`filter(None, …)` ou prédicats).
- `map()` pour appliquer une fonction de normalisation à chaque enregistrement (chaînes, casse, formats).
- Le `match-case` au lieu des `if-elif-else` pour router selon le format : chaîne brute, dictionnaire, tuple de connexion.

Ce qu'on présente, dans l'ordre

1. **Une chaîne construite sans rien lire** — chaque `map` / `filter` enveloppe l'itérateur précédent ; rien ne circule avant la boucle `for`.
2. **map(clean_raw_line)** — retire espaces, \r, codes couleur et caractères invisibles ; `isprintable()` évite ce travail sur les 95 % de lignes déjà propres.
3. **filter et ses prédicats** — `filter(tracker.keep_line)` supprime vides, commentaires et stack traces **en comptant la cause** ; `filter(None, …)` vide la liste noire de ses lignes vides ; une `lambda` filtre par niveau.
4. **match-case** — `parse_line` reconnaît la **forme** : un dict JSON (motif imbriqué), un tuple pare-feu (`"FW", …, *rule`), sinon une chaîne applicative.
5. **map(normalize_record)** — niveaux (`case "WARN" | "WRN"`), utilisateurs (regex), 5 formats de dates, IP validées, JSON et encodage réparés.
6. **filter(tracker.accept)** — écarte seulement l'inexploitable (IP ou date impossible, ligne tronquée) et note la raison.

Le code à montrer (parsing.py + main.py)

parsing.py · main.py

```
match decode_line(line):
    case {"ts": ts, "lvl": str(level), "svc": str(service),
          "client": {"ip": ip, "port": port, "proto": proto}, **rest}:
        ...                                  # dictionnaire JSON
    case ("FW", ts, action, proto, src_ip, src_port, dst_ip, dst_port, *rule):
        ...                                  # tuple pare-feu
    case str() as text:
        return _parse_app_line(text)         # chaîne brute

records = map(normalize_record, map(parse_line, lines))
records = filter(tracker.accept, records)
records = filter(lambda r: LEVEL_RANK[r["level"]] >= min_rank, records)
```

2

Nettoyage : ce que ça donne

Réparer tout ce qui peut l'être, écarter le reste, et compter chaque décision

68,3 M
enregistrements valides

1,3 M
corrections : lignes sauvées

1,0 M
lignes écartées

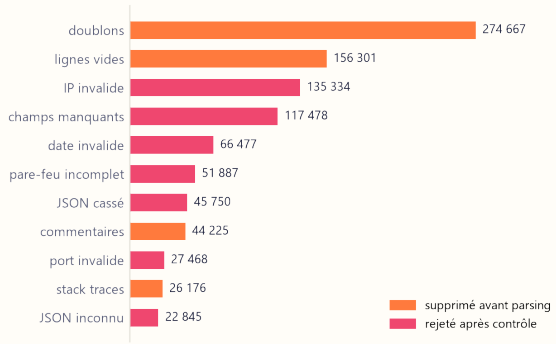
1,4 %
seulement de pertes

Avant / après : une vraie ligne passée dans notre code

entrée sale

```
01/10/2026 08:15:32 | err | API_GATEWAY | 88.12.34.56:51432 | user=USER_7 | event=http_request |
    {'status':'500','latency_ms':-1}
```

timestamp	2026-10-01 08:15:32+00:00	date réécrite en ISO, UTC
level	ERROR	« err » normalisé (match-case)
service	api-gateway	casse et « _ » corrigés
connection	('88.12.34.56', 51432, 'TCP')	tuple, TCP par défaut
user	u7	« USER_7 » ramené à « u7 »
payload	{'status': 500, 'latency_ms': None}	guillemets et latence corrigés



QUESTIONS PROBABLES DU JURY

Q Pourquoi match-case plutôt que if / elif ?
On décrit la forme attendue (clés, longueur, types) et on capture les valeurs en même temps, au lieu d'enchaîner isinstance, len et des extractions à la main.

Q À quoi sert filter(None, ...) ?
Il ne garde que les éléments « vrais » : les chaînes vides disparaissent.

Q Pourquoi passer une méthode à filter ?
Une fonction est un objet : tracker.accept décide de garder la ligne ET compte la cause du rejet.

Q Réparer ou rejeter, comment décidez-vous ?
Récupérable (casse, format) : on corrige. Impossible (IP 999.x, 13e mois) : on rejette et on note pourquoi.

TRANSITION Les données sont propres et typées : on peut les compter et y chercher des attaques → Module 3

MODULE 3 · CE QU'ON PRÉSENTE

3 Agrégation & détection

set, frozenset, defaultdict, et une boucle qui démasque la force brute

CE QUE DEMANDE L'ÉNONCÉ

- Un `set` des IP uniques ayant généré des erreurs.
- Les opérations `|` `&` `-` `^` entre la liste noire (`frozenset` immuable) et les IP actives.
- `defaultdict` pour compter par catégorie ou par utilisateur sans `KeyError`.
- Une détection de force brute combinant `while` et `for`, avec `break`, `continue` et la clause `else` des boucles.

Ce qu'on présente, dans l'ordre

1. **Une seule passe** — pour chaque ligne, `stats.update` met à jour des compteurs : on ne garde jamais les lignes elles-mêmes.
2. **defaultdict** — `compteur[cle] += 1` crée le 0 tout seul, donc jamais de KeyError ; version imbriquée service → niveau, et `defaultdict(set)` IP → ports.
3. **set et frozenset** — `error_ips.add(ip)` ignore les doublons : on obtient directement les IP uniques. La liste noire est un frozenset : personne ne peut la modifier pendant l'analyse.
4. **Le tuple comme clé** — `blocked_flows[(ip, port, proto)] += 1` compte les flux bloqués ; regroupés par IP, ils révèlent les scans de ports.
5. **Force brute** — une deque garde les échecs des 2 dernières minutes par IP ; un `while` retire les trop anciens ; à la fin, une boucle `for` tranche (code ci-dessous).

Le code à montrer (detection.py)

summarize()

```
for moment, user, ok in suspect_events:
    if not ok:
        consecutive += 1
        continue            # échec suivant
    if consecutive >= 5 and récent:
        compromis = user
        break               # attaque réussie
    consecutive = 0
else:
    statut = "bloquée"      # aucun break
```

Les opérations d'ensembles

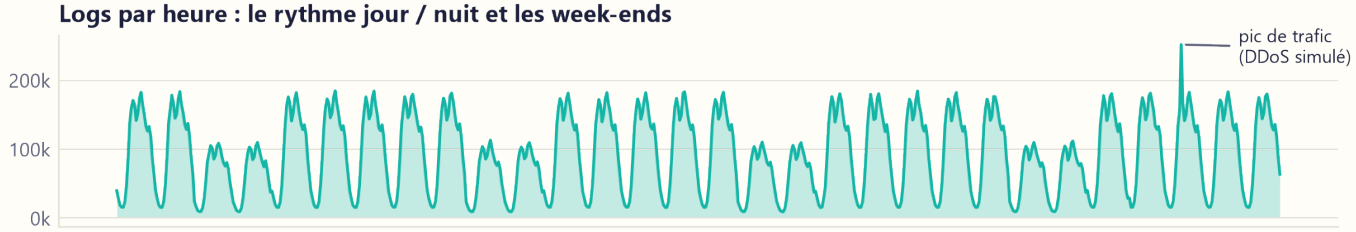
`actives | bl`	toutes les IP connues	17 838
`actives & bl`	listées et actives	190
`erreurs & bl`	listées, en erreur	190
`actives - bl`	actives non listées	17 348
`bl - actives`	listées jamais vues	300
`actives ^ bl`	dans l'un OU l'autre	17 648

3

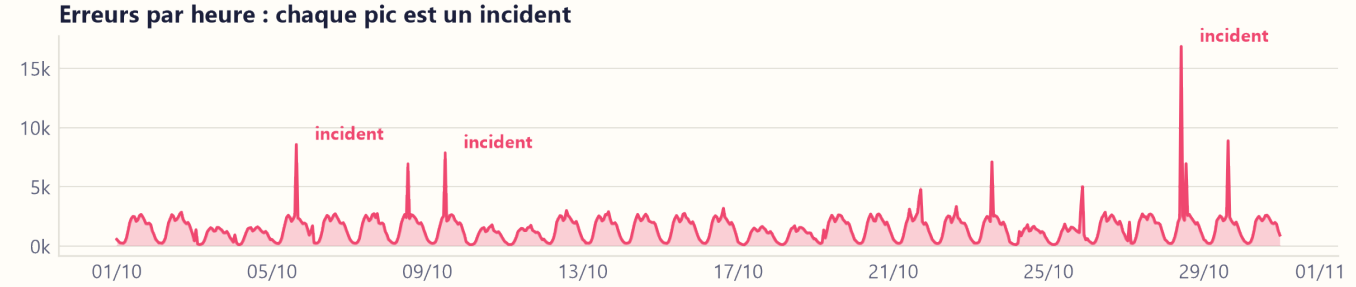
Agrégation : ce que ça donne

68 millions de lignes résumées en quelques compteurs

Logs par heure : le rythme jour / nuit et les week-ends



Erreurs par heure : chaque pic est un incident



190
IP en liste noire actives sur 490

30
scanners (15 ports visés ou plus)

60
IP en force brute

48
ont réussi, confirmé par 105 élévations

QUESTIONS PROBABLES DU JURY

Q **Pourquoi un frozenset pour la liste noire ?**
Elle ne doit pas changer pendant l'analyse ; un frozenset ne peut pas être modifié, et le test « ip in blacklist » reste instantané.

Q **Set ou defaultdict, quelle différence ?**
Le set répond à « quelles IP ? » (unicité), le defaultdict à « combien de fois ? » (comptage).

Q **À quoi sert le else d'une boucle for ?**
Il s'exécute seulement si la boucle va au bout sans break : aucune connexion n'a réussi, l'attaque a échoué.

Q **Pourquoi une deque pour la fenêtre ?**
On ajoute à droite et on retire à gauche en temps constant : c'est fait pour une fenêtre glissante.

Q **Pourquoi 60 attaquants mais 48 réussites ?**
Les incidents sont regroupés par IP sur les 30 jours ; les élévations de privilèges confirment indépendamment les connexions réussies.

TRANSITION On sait compter et détecter : reste à rendre la sécurité configurable et observable → Module 4

MODULE 4 · CE QU'ON PRÉSENTE

4 Règles & introspection

apply_security_rules(*args, **kwargs), des fonctions comme objets, et globals() / locals()

CE QUE DEMANDE L'ÉNONCÉ

- Une fonction maître `apply_security_rules(*args, **kwargs)` : un nombre variable de fonctions de sécurité en `*args`, une configuration dynamique en `**kwargs`.
- Les fonctions manipulées comme des objets de première classe (passées en argument à d'autres fonctions).
- Un audit qui inspecte l'exécution avec `globals()` et `locals()` et exporte un rapport des variables.

Ce qu'on présente, dans l'ordre

1. **Une règle = une fonction** — `rule(record, **config)` renvoie une alerte ou None. Nous en avons 9 : injection SQL, XSS, liste noire, port exposé…
2. ***args et **kwargs** — `*args` regroupe les règles dans un **tuple**, `**kwargs` la configuration dans un **dict** ; à l'appel, `*DEFAULT_RULES` « déballe » le tuple.
3. **Une fonction qui renvoie une fonction** — on prépare le moteur une seule fois (`filter` + `lambda` retire les règles désactivées) et on récupère `check(record)`, appelée sur chaque ligne. C'est une **fermeture**.
4. **Robustesse** — une règle qui plante est isolée par `try / except / else` ; un argument qui n'est pas une fonction lève `TypeError`.
5. **Introspection** — `main.py` appelle `export_audit_report(path, globals(), locals())` : chaque variable est décrite (type, taille, aperçu) dans audit.json.

Le code à montrer (logengine/rules.py)

rules.py

```python
def apply_security_rules(*args: Rule, **kwargs: Any) -> RuleChecker:
    for rule in args:
        if not callable(rule):
            raise TypeError("une règle doit être une fonction")
    disabled = kwargs.get("disabled", ())
    active = tuple(filter(lambda rule: rule.__name__ not in disabled, args))

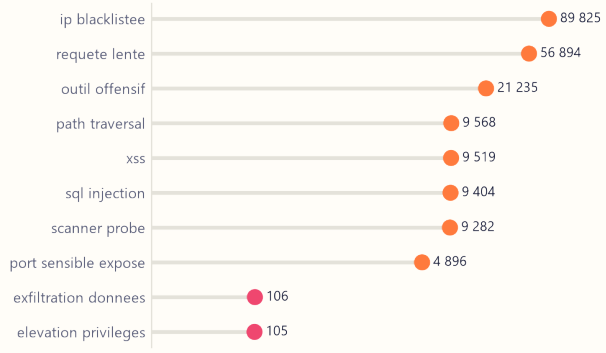
    def check(record: LogRecord) -> list[Alert]:
        alerts = []
        for rule in active:
            try:
                alert = rule(record, **kwargs)    # on retransmet **kwargs
            except Exception as exc:
                print(f"{rule.__name__} a échoué")
            else:
                if alert is not None: alerts.append(alert)
        return alerts
    return check                                  # la fonction elle-même
```

MODULE 4 · RÉSULTATS

4 Règles : ce que ça donne

Des alertes sur 12 Go, un audit de l'exécution, et la démo en direct

210 834 alertes par règle



Extrait d'audit.json (locals())

blacklist — frozenset · 32 984 octets

```
frozenset({'102.12.183.222', '102.125.28.142', '102.
```

check_rules — function · 152 octets

```
<function apply_security_rules.<locals>.check at 0x0
```

records — filter · 48 octets

```
<filter object at 0x000001A50DC3F280>
```

stats — LogStatistics · 56 octets

```
<logengine.aggregation.LogStatistics object at 0x000
```

records ne pèse que quelques dizaines d'octets : c'est un itérateur, il ne contient pas les données.

démo temps réel

```
$ python main.py live
[live] 2,357 lignes lues, 7 alertes
[HIGH] xss  217.135.14.178 javascript:..
[FORCE BRUTE] 92.112.165.218 : 5 échecs
```

QUESTIONS PROBABLES DU JURY

Q **Quelle différence entre *args et **kwargs ?**
*args rassemble les arguments positionnels dans un tuple, **kwargs les arguments nommés dans un dict.

Q **Pourquoi apply_security_rules renvoie-t-elle une fonction ?**
*args ne contient que des règles, pas la ligne à analyser : on prépare le moteur une fois, puis check(record) tourne sur 69 M de lignes.

Q **Qu'est-ce qu'une fermeture (closure) ?**
Une fonction interne qui garde accès aux variables de la fonction qui l'a créée (ici active et kwargs), même après sa fin.

Q **Pourquoi globals() / locals() sont-ils appelés dans main.py ?**
Appelés dans audit.py, ils décriraient le module d'audit, pas le programme qu'on veut inspecter.

Q **Comment ajouter une nouvelle règle ?**
Écrire def rule_x(record, **config) et l'ajouter à DEFAULT_RULES. Rien d'autre à changer.

TRANSITION — Le moteur est complet : place au bilan qualité et à la démonstration → Conclusion

5

CONCLUSION

Qualité, barème et démo

Chaque critère du barème et l'endroit où le montrer

Environnement & livrables	venv, requirements.txt (dépendances directes), scripts .py uniquement, rapport PDF, Git	2 pts
Mémoire & générateurs	yield, yield from, follow_log_file · ~19 Mo pour 2 M de lignes	4 pts
Structures de données	set, frozenset, defaultdict, tuple clé de dict, match-case	4 pts
Programmation fonctionnelle	map, filter, lambda, méthodes passées à filter, fonction renvoyée	4 pts
Règles & introspection	apply_security_rules(*args, **kwargs), globals() / locals()	3 pts
Qualité & robustesse	PEP 8, mypy 0 erreur, 75 tests, try/except/else, for-else	3 pts

75
tests pytest, tous verts

0
erreur mypy (code et tests)

0
écart PEP 8 (pycodestyle)

17 min
pour analyser les 12 Go

EN UNE PHRASE

Un moteur qui traite **12 Go de logs hétérogènes et dégradés** avec quelques dizaines de mégaoctets de mémoire, répare l'essentiel des défauts, retrouve les incidents et les attaques injectés, et fonctionne aussi **en temps réel**.

POUR ALLER PLUS LOIN

Analyse parallèle (un processus par fichier, puis fusion des compteurs) pour diviser le temps par 5 ou 6 ; **séparer les campagnes** d'une même IP attaquante ; **rattacher les stack traces** à la ligne d'erreur qui les précède.

DÉMO

```
python main.py demo      python main.py live      python -m pytest
```

Code complet : https://github.com/Babacar2804/Projet-PythonSecure-log-anlysis